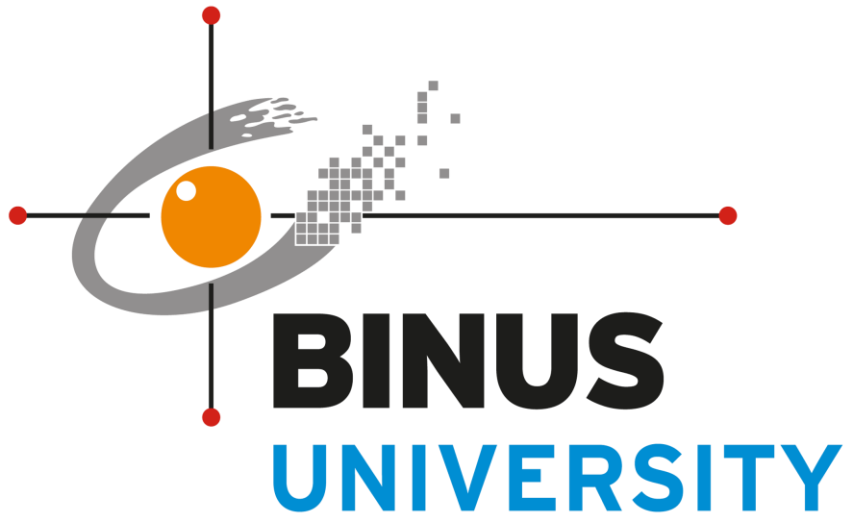


Clustered Face Identification



Rafi Hazel Tafara – 2702347263

Felix Young – 2702274513

Marcellino Asanuddin – 2702242650

BINUS UNIVERSITY

2025

I. Latar Belakang

Dalam manajemen media, tidak jarang diperlukan teknologi untuk mencari dan mengenali suatu orang dalam banyak foto/video. Semakin banyak foto/video yang ada, semakin lama pula waktu yang dibutuhkan untuk menyortir *file* secara manual. Karena itu, teknologi *face recognition* digunakan untuk mengotomatisasi pekerjaan tersebut.

Di masa ini, sudah terdapat banyak teknologi *face recognition* yang bisa digunakan secara bebas dengan performa yang bervariasi. Akan tetapi, teknologi ini umumnya terbatas pada *input* yang berupa foto saja, sehingga implementasi untuk *input* yang berupa video memerlukan pemrosesan tambahan. Terdapat aspek-aspek tertentu yang hanya dimiliki video yang bisa dimanfaatkan, salah satunya melalui *object tracking* untuk melacak pergerakan wajah yang melewati beberapa *frame*. Melalui ini, dua gambar wajah dengan *tracking ID* yang sama bisa dipastikan mengacu pada orang yang sama tanpa melalui teknologi *face recognition* konvensional.

Walaupun terdapat beberapa sistem *face recognition* untuk video yang juga cukup populer, kebanyakan sistem tersebut bersifat *closed source* yang kodenya tidak bisa dilihat atau digunakan secara bebas. Teknologi yang bersifat *closed source* tersebut tentunya tidak dapat digunakan oleh orang-orang yang ingin merakit solusi *face recognition* yang diadaptasikan dengan kebutuhan tertentu secara mandiri.

Proyek ini berfokus pada pembuatan *pipeline* untuk mengenali wajah orang-orang yang ada di sebuah *input* video. Aplikasi yang kami buat memanfaatkan kombinasi teknologi *face detection*, *face tracking*, dan *clustering* yang semuanya (kode, model, *dataset*) bersifat *open source* sehingga bisa digunakan dan diadaptasikan oleh siapa pun.

II. Dataset

Kami melatih model YOLOv11l dari Ultralytics untuk melakukan deteksi dan pelacakan wajah pada video. Dataset yang digunakan diambil dari dua sumber yang semuanya berasal dari Kaggle.

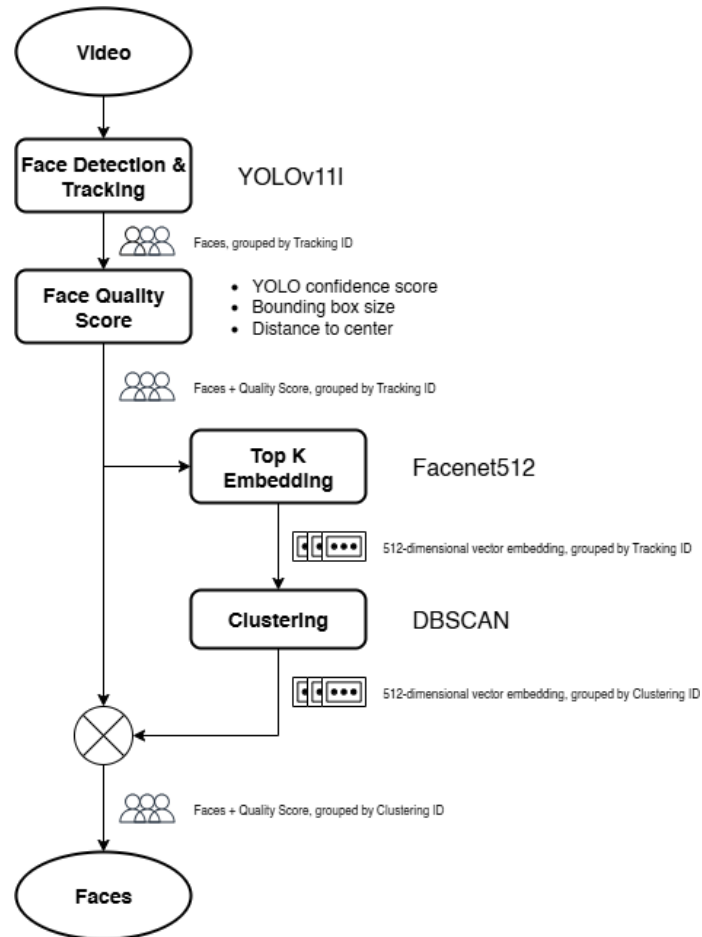
1. [Face Detection Dataset \(Sudanshu Rastogi\)](#), lisensi Apache 2.0
2. [Face-Detection-Dataset \(Fares Elmenshawii\)](#), lisensi CC0: Public Domain

Semua dataset berisi foto JPG yang diberi *annotation* berupa *bounding box* yang menunjukkan lokasi wajah pada foto. Dataset pertama berisi total 16.304 foto yang terbagi menjadi 12.880 foto training, 3.226 foto validation, dan 198 foto test. Dataset kedua berisi total 16.733 foto yang terbagi menjadi 13.386 foto training dan 3.347 foto validation.

Terdapat beberapa penyesuaian dataset yang dilakukan sebelum melakukan training. Pertama, dataset *test* dari dataset pertama tidak digunakan karena tidak adanya file label untuk foto-foto tersebut. Kedua, label bounding box di dataset pertama ditulis dalam format xywh, sedangkan YOLO mengharuskan label bounding box ditulis dalam format xywh yang dinormalisasikan. Normalisasi ini telah dilakukan melalui Jupyter Notebook yang ada di dalam repositori.

III. Arsitektur Model

Dalam proyek ini, proses pengenalan wajah dalam video membutuhkan empat langkah: *Face Detection & Tracking*, *Face Quality Score*, *Top K Embedding*, dan *Clustering*. Detail *pipeline* dapat dilihat pada gambar di bawah.



Gambar 1: Diagram arsitektur *Clustered Face Identification*

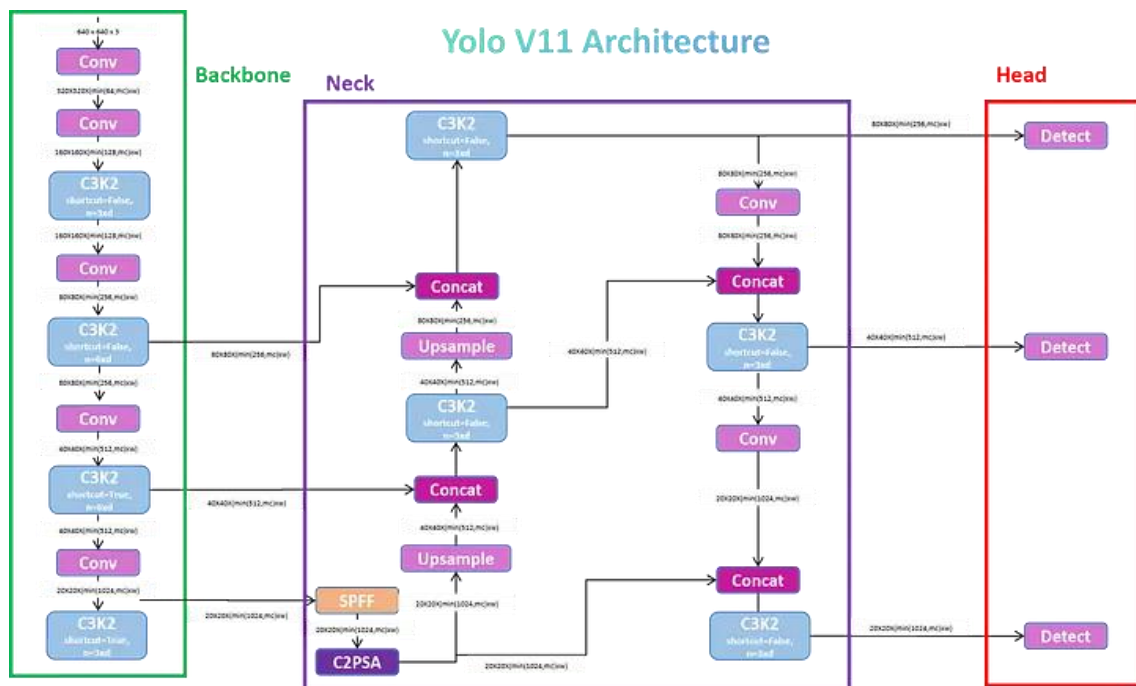
1. Face Detection & Tracking

Pada tahap ini, model akan mendeteksi wajah sekaligus melakukan pelacakan wajah *frame-by-frame*. Masing-masing wajah akan diberi *Tracking ID*, dan idealnya setiap wajah yang berbeda akan memiliki ID yang berbeda juga.



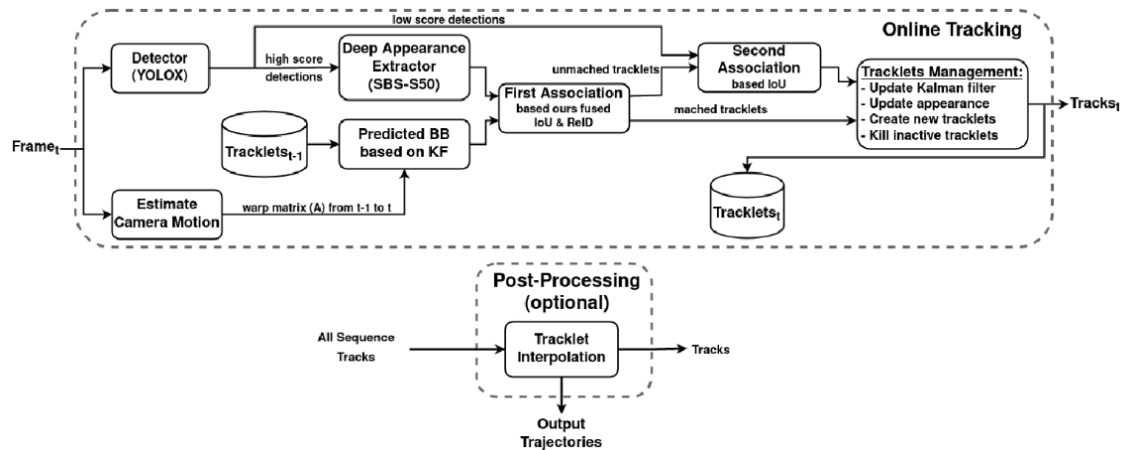
Gambar 2: Contoh input video

Model YOLOv11 dipilih karena kemudahan proses *training*, kualitas inferensi, serta fitur *object tracking* yang sudah terintegrasi dengan *library* nya.

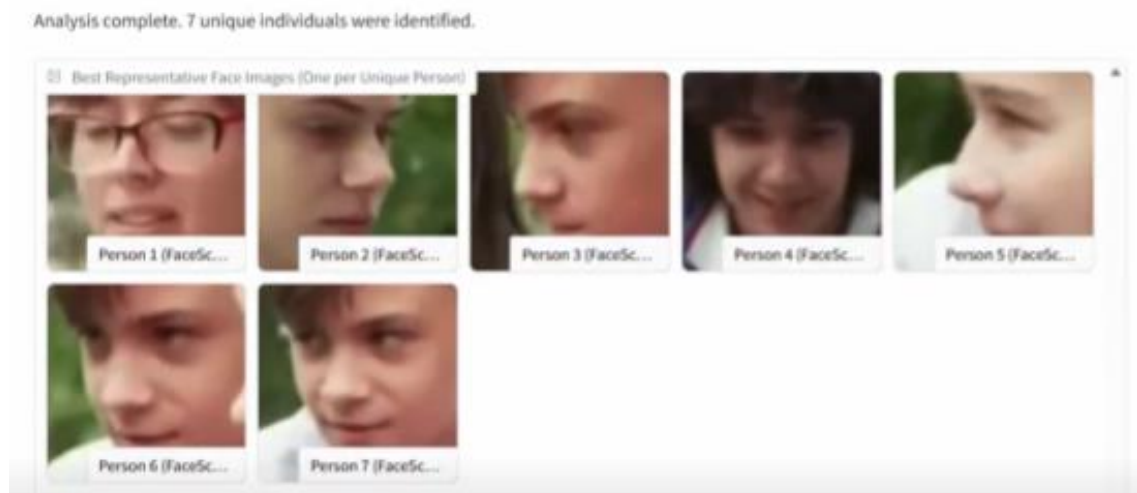


Gambar 3: Arsitektur model YOLOv11

Model sebelumnya dilatih menggunakan dataset pilihan yang sudah dijelaskan di Bab II, dan *tracking* dilakukan dengan algoritma BoTSort.



Gambar 4: Flowchart algoritma BoTSort



Gambar 5: Wajah-wajah yang berhasil diidentifikasi dalam video

Parameter *conf* dan *iou* dari YOLO bisa diatur sesuai keinginan pengguna.

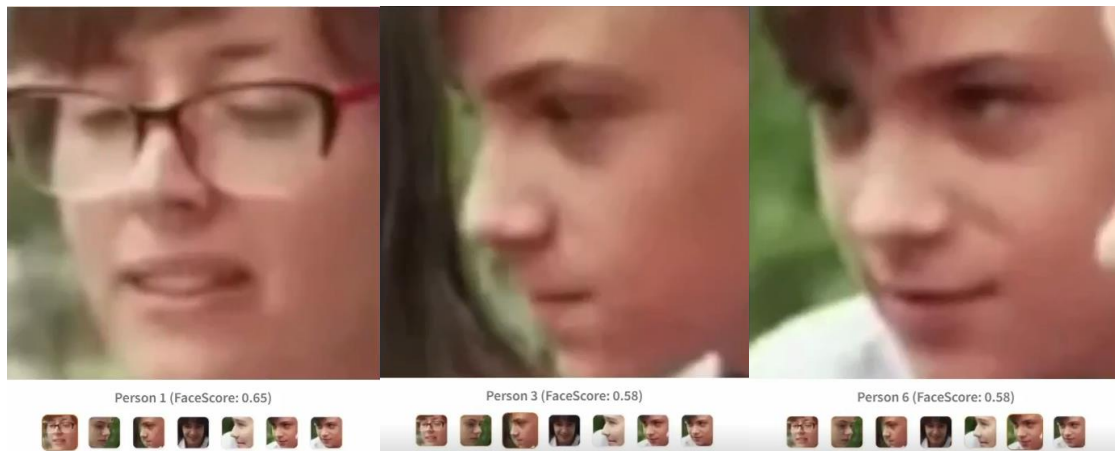
2. Face Quality Score

Setiap *tracking ID* akan memiliki beberapa bounding box wajah yang melewati beberapa frame video. Tidak semua gambar wajah akan digunakan untuk menghasilkan embedding untuk menghemat waktu. Karena itu, wajah dengan kualitas terbaik saja yang akan digunakan untuk menghasilkan embedding.

Terdapat tiga kriteria untuk menentukan skor kualitas wajah yang digunakan dalam proyek ini, yakni:

- Nilai *confidence* dari deteksi wajah YOLO, yang mengacu pada seberapa yakin model YOLO merasa gambar yang dideteksi merupakan wajah.
- Ukuran bounding box, di mana bounding box yang berukuran besar umumnya berarti wajah yang lebih dekat ke kamera, yang berarti wajah terlihat lebih jelas.

- Jarak dari titik tengah frame, di mana wajah yang berada di area tengah frame diberi skor lebih tinggi. Wajah yang berada di pinggir frame seringkali akan terpotong ketika melakukan *face alignment* di tahap berikutnya.

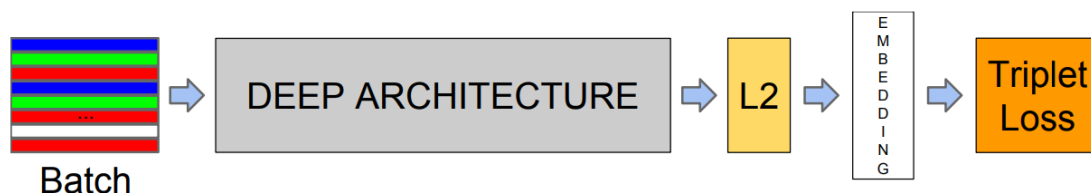


Gambar 6: Tampak *close-up* beberapa wajah beserta skor kualitas masing-masing

Pengguna bisa mengatur bobot untuk masing-masing kriteria di dalam implementasi.

3. Top K Embedding

Setelah semua wajah diberi skor kualitas, K gambar wajah dengan skor tertinggi untuk masing-masing *tracking ID* akan digunakan untuk menghasilkan embedding yang kemudian dirata-ratakan; gambar wajah yang lain cukup diabaikan saja. Angka K merupakan parameter yang bisa diatur oleh pengguna.

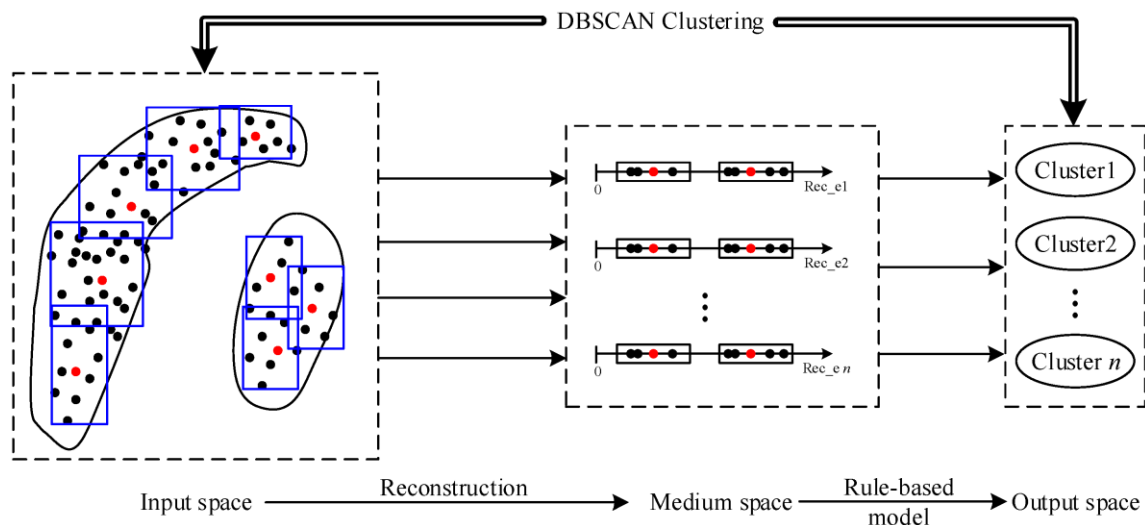


Gambar 7: Arsitektur algoritma FaceNet

Model embedding yang digunakan dalam proyek ini adalah Facenet512, yang dipilih karena akurasi yang cukup tinggi dibandingkan model embedding serupa. Facenet512 menggunakan DCNN yang diikuti oleh regularisasi L2 untuk menghasilkan embedding wajah dan triplet loss untuk memperkecil jarak wajah yang serupa (*anchor and positive*) dan memperbesar jarak wajah yang berbeda (*anchor and negative*).

Sebelum gambar wajah dimasukkan ke dalam model embedding, gambar wajah perlu dilakukan *alignment* terlebih dahulu agar hasil embedding lebih konsisten. Proses *alignment* dilakukan dengan melakukan transformasi pada frame mengikuti *5-point landmarks* pada wajah yang diprediksi menggunakan model *shape_predictor_5_face_landmarks* dari library dlib di Python. Gambar yang sudah ditransformasi ini kemudian di-*crop* ke dalam ukuran 160x160.

4. Clustering



Gambar 8: Arsitektur algoritma *clustering* DBSCAN

Embedding yang sudah dihasilkan dari langkah sebelumnya akan dikelompokkan menggunakan algoritma DBSCAN (Density-Based Spatial Clustering of Applications with Noise). Tahap ini bertujuan mengkategorisasi ulang wajah mirip yang diberi *tracking ID* yang berbeda di tahap pertama menjadi satu *cluster*. DBSCAN akan menghasilkan *cluster-cluster* yang berdasarkan *cosine similarity* dari masing-masing embedding, di mana masing-masing *cluster* merujuk pada satu orang yang sama.

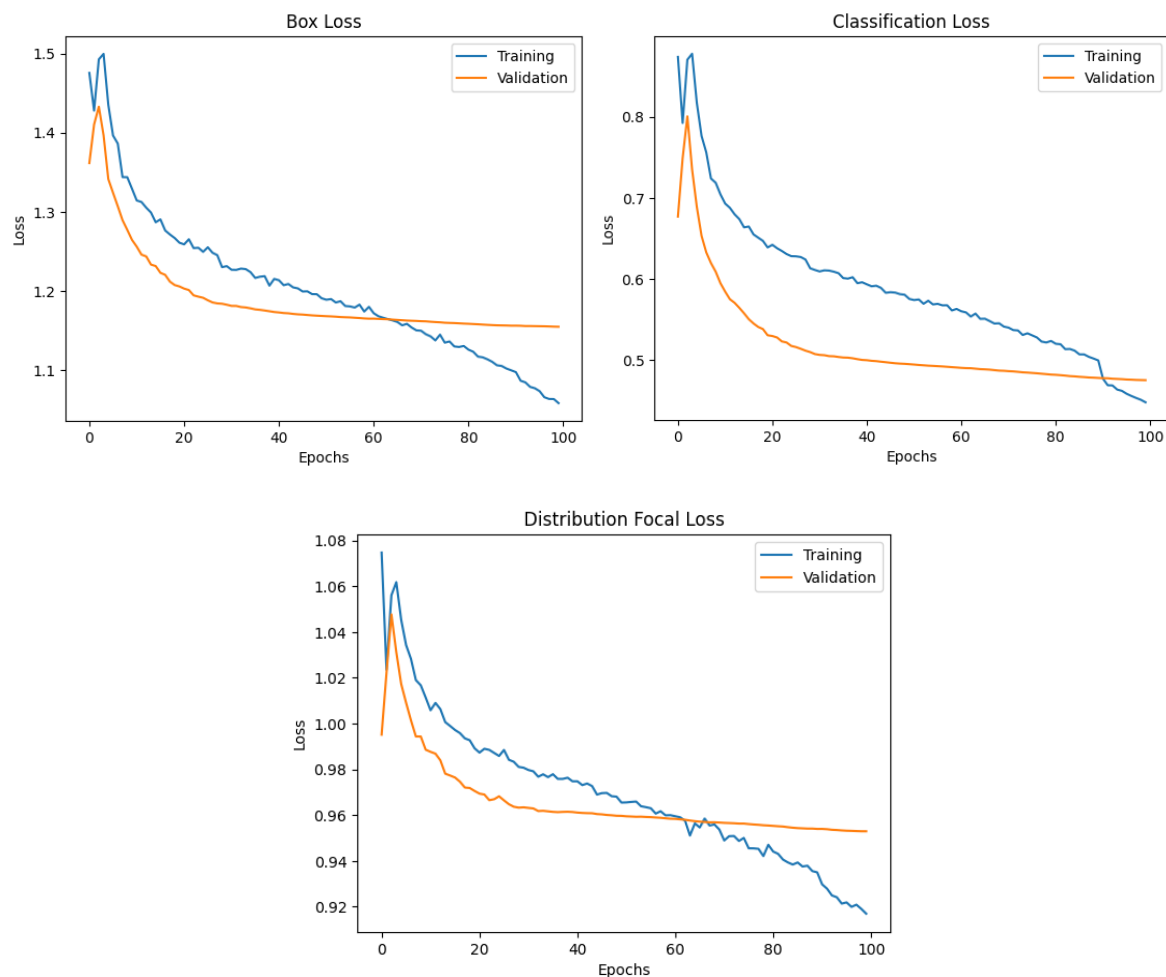
Parameter epsilon dari DBSCAN bisa diatur oleh pengguna jika diinginkan.

IV. Evaluasi

Proses pelatihan model deteksi wajah YOLO dilakukan selama 100 epoch dengan *hyperparameter* sebagai berikut:

- Batch Size: 16
- Target Image Size: 320
- IOU: 0.7
- Patience: 10

Berikut merupakan grafik *box loss*, *classification loss*, dan *distribution focal loss* selama *training* dan *validation*.



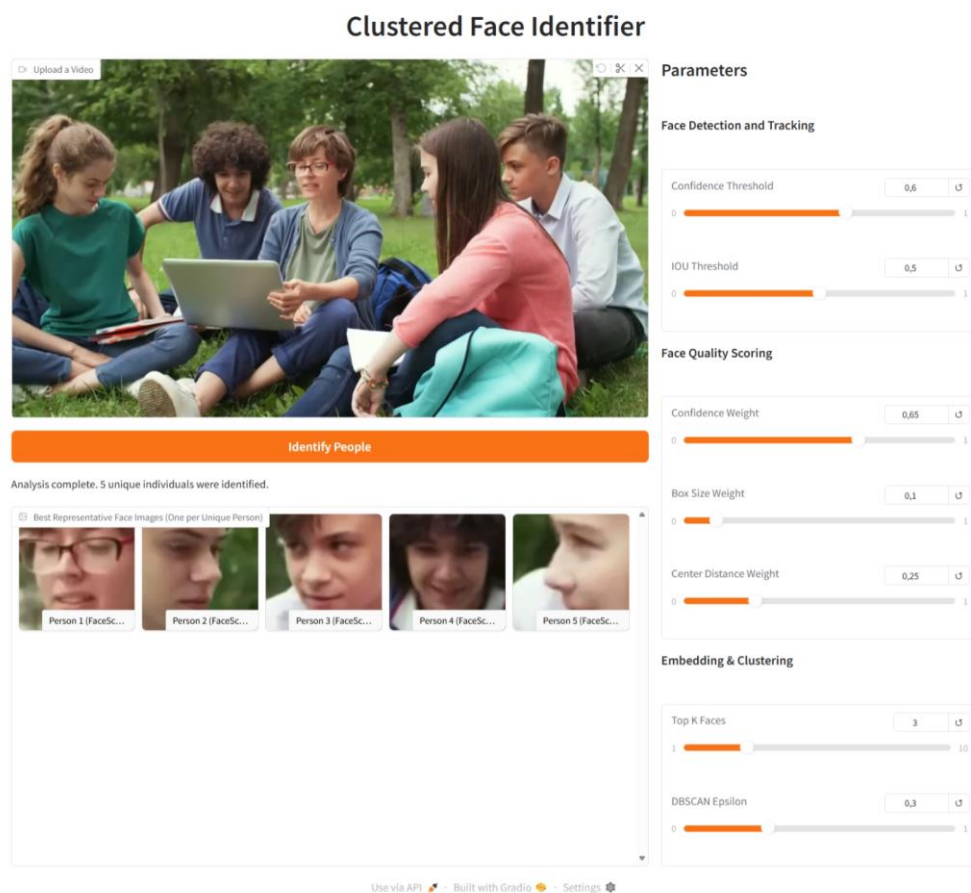
Gambar 9: Grafik *training* dan *validation loss*

Berdasarkan grafik tersebut, dapat disimpulkan bahwa model dilatih sampai di sekitar ambang batas *overfitting*.

V. Deployment

Dengan *library* Gradio, kami membuat sebuah tampilan *frontend* yang terhubung dengan implementasi pengenalan wajah dalam video yang di-*upload* oleh pengguna. Pengguna juga bisa mengatur parameter seperti *confidence threshold*, *IOU threshold*, *confidence weight*, *box size weight*, *center distance weight*, *top K faces*, dan *DBSCAN epsilon* untuk mengatur kinerja *pipeline* di balik layar. Kegunaan masing-masing parameter sudah dijelaskan di Bab III.

Berikut ini adalah tampilan UI dari proyek ini.



Gambar 10: Tampilan UI aplikasi (Gradio)

Untuk menjalankan tampilan *frontend* tersebut, setelah mengunduh repositori dan menginstal dependensi yang dibutuhkan dengan `pip install -r requirements.txt`, jalankan `python app/app.py` di *root folder*. Tampilan *frontend* akan bisa diakses lewat browser di URL lokal yang diberikan program, yang *default*-nya berada di `https://localhost:7860`.

VI. Refleksi

Proyek ini berhasil menyediakan *pipeline* untuk melakukan identifikasi wajah orang dalam sebuah video yang sepenuhnya *open-source*.

Kunci dari *pipeline* pada proyek ini terletak pada pemanfaatan kombinasi teknologi *face detection*, *face tracking*, *embedding* dan *clustering*. Aspek ini memenuhi motivasi awal proyek untuk menciptakan sistem yang tidak hanya berkinerja baik, tetapi juga mudah diadaptasi dan diimplementasikan secara mandiri oleh pengembang, mengatasi keterbatasan yang dimiliki oleh solusi *closed-source*.

Dalam hal implementasi, kami menggunakan *library* Gradio untuk mengembangkan *frontend* yang rapi dan intuitif. Kami menambahkan fitur untuk mengatur parameter *pipeline* secara langsung seperti *confidence threshold*, *IOU threshold*, *DBSCAN epsilon*, dan lain-lain. Dengan memberikan kontrol penuh kepada pengguna, diharapkan kualitas dapat meningkat sesuai dengan karakteristik video yang mereka miliki.

Secara keseluruhan, proyek ini menunjukkan bahwa solusi *face recognition* yang *open-source* dapat menjadi alternatif bagi yang menginginkan fleksibilitas. Tantangan utama yang dihadapi selama pengerjaan proyek ini adalah memikirkan cara-cara kreatif untuk mengoptimalkan akurasi *pipeline* dalam menentukan apakah dua wajah mengacu pada orang yang sama ataupun beda. Hal tersebut dijawab dengan melakukan kombinasi *tracking* dan *clustering* pada *embedding* sehingga kekurangan di salah satu tahap bisa ditutupi di tahap lainnya.

Referensi

- Rao, S. N. (2024, October 22). YOLOv11 architecture explained: Next-level object detection with enhanced speed and accuracy. Medium. <https://medium.com/@nikhil-rao-20/yolov11-explained-next-level-object-detection-with-enhanced-speed-and-accuracy-2dbe2d376f71>
- Khanam, R., & Hussain, M. (2024, October 23). YOLOv11: An overview of the key architectural enhancements (arXiv:2410.17725v1). ArXiv. <https://arxiv.org/html/2410.17725v1>
- Ultralytics. (n.d.). YOLO11 - Ultralytics YOLO Docs. Retrieved January 11, 2026, from <https://docs.ultralytics.com/models/yolo11/>
- Aharon, N., Orfaig, R., & Bobrovsky, B.-Z. (2022). BoT-SORT: Robust Associations Multi-Pedestrian Tracking (arXiv:2206.14651). arXiv. <https://doi.org/10.48550/arXiv.2206.14651>
- Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. In Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD '96) (pp. 226–231). ACM. <https://dl.acm.org/doi/10.5555/3001460.3001507>
- Zhang, X., Shen, X., & Ouyang, T. (2022). Extension of DBSCAN in online clustering: An approach based on three-layer granular models. Applied Sciences, 12(19), 9402. <https://doi.org/10.3390/app12199402>
- Schroff, F., Kalenichenko, D., & Philbin, J. (2015). FaceNet: A unified embedding for face recognition and clustering (arXiv:1503.03832). arXiv. <https://doi.org/10.48550/arXiv.1503.03832>